



Python I - Fundamentos

Python I – Introdução ao curso

Formação inicial: Fundamentos da linguagem

Carga horária: *36 horas

Objetivos:

Este curso tem como objetivo desenvolver profissionais para compreender a linguagem de programação Python, desenvolver algoritmos e utilizar módulos de terceiros para extração de dados.



Apresentação e Expectativas

Expectativa:

1 -

Senac



Pasta compartilhada

https://drive.google.com/drive/folders/1v9TI1u1j11BiqSowgHY2nOgbtZe_BMDh?usp=sharing

WhatsApp

O que Python?

Expectativa:

1 -



Histórico

Quem criou? Van Rossum

Quando? O Python foi criado em 1989 e lançado em 1991 como nome de Python

Com qual motivação? Precisava de uma linguagem de script que tivesse sintaxe semelhante ao **ABC** e que tivesse acesso às chamadas de sistema;

Principais Características:

- Linguagem simples em relação a outras linguagens como C e C++;
- Tem suporte a orientação a objetos;
- Interpretada por uma máquina virtual;
- Tipagem dinamicamente forte;
- Multiplataforma (funciona em sistemas Windows, Linux e Mac);



Interpretador || Compilador



CPython é a implementação principal da linguagem de programação Python, escrita em Linguagem C. CPython é um interpretador de Bytecode.



PyPy é um compilador **Just in Time (JIT)**, ele usa uma técnica conhecida como meta-tracing, que é responsável por **transformar um interpretador em um compilador JIT**.



Jython é uma implementação da linguagem Python que **gera bytecode** para máquinas Java (JVM - Java Virtual Machine). Com ela é possível fazer o desenvolvimento de aplicações híbridas que unem código em Java e Python.



IronPython é uma implementação da linguagem de programação Python escrita em C#, para plataforma.NET e Mono, criada por Jim Hugunin.

Versões/Documentação

Python » English » 3.14.2 » 3.14.2 Documentation »

Download

Download these documents

Docs by version

- Python 3.15 (in development)
- Python 3.14 (stable)
- Python 3.13 (stable)
- Python 3.12 (security-fixes)
- Python 3.11 (security-fixes)
- Python 3.10 (security-fixes)
- Python 3.9 (EOL)
- Python 3.8 (EOL)
- Python 3.7 (EOL)
- Python 3.6 (EOL)
- Python 3.5 (EOL)
- Python 3.4 (EOL)
- Python 3.3 (EOL)

Python 3.14.2 documentation

Welcome! This is the official documentation for Python 3.14.2.

Documentation sections:

[What's new in Python 3.14?](#)
Or all "What's new" documents since Python 2.0

[Installing Python modules](#)
Third-party modules and PyPI.org

[Tutorial](#)
Start here: a tour of Python's syntax and features

[Distributing Python modules](#)
Publishing modules for use by other people

[Library reference](#)
Standard library and builtins

[Extending and embedding](#)
For C/C++ programmers

<https://docs.python.org/3.14/>

Outros links:

<https://www.python.org/dev/peps/pep-0008/>

<https://wiki.python.org.br/GuiaDeEstilo>

<https://www.w3schools.com/python/>

python™

Donate Search GO Socialize

About Downloads Documentation Community Success Stories News Events

Download the latest version for Windows

[Download Python install manager](#)

Or get the standalone installer for [Python 3.14.2](#)

Looking for Python with a different OS? Python for [Windows](#), [Linux/Unix](#), [macOS](#), [Android](#), [other](#)

Want to help test development versions of Python 3.15? [Pre-releases](#), [Docker images](#)



<https://www.python.org/downloads/>

Para que serve o Python?

Construção coletiva:

1 -



Para que serve o Python?

Python

Data Science

Pandas

NumPy

Scikit Learn, TensorFlow, Keras

Automação

Extração de dados

Selenium

Pytest, PyAutoGui

BeautifulSoup

WebDriver

Desenvolvimento

Flet

Flask

Django

Tkinter

Desenvolvimento Game

PyGame

PyOpenGL

Kivy

Ethical Hacker

Scapy

Flowgrep

Subrute

Mão na massa...

Ambientes utilizando Nuvem:

✓ Google Colab: <https://colab.research.google.com/>

✓ Repl.it: <https://replit.com/>

Ambientes Locais:

✓ Realizar a instalação do **VS Code**
<https://code.visualstudio.com/download>

✓ Utilizar o interpretador **Python 3.10**
<https://ideasdays.com/python/pyth.html>





P OO – Programação Orientação a Objetos

Paradigmas de programação:

Paradigma de programação é um meio de se classificar as linguagens de programação baseado em suas funcionalidades. As linguagens podem ser classificadas em vários paradigmas. O paradigma de programação fornece a visão que o programador possui sobre a estruturação e execução do programa.

Mais informações: https://pt.wikipedia.org/wiki/Paradigma_de_programa%C3%A7%C3%A3o

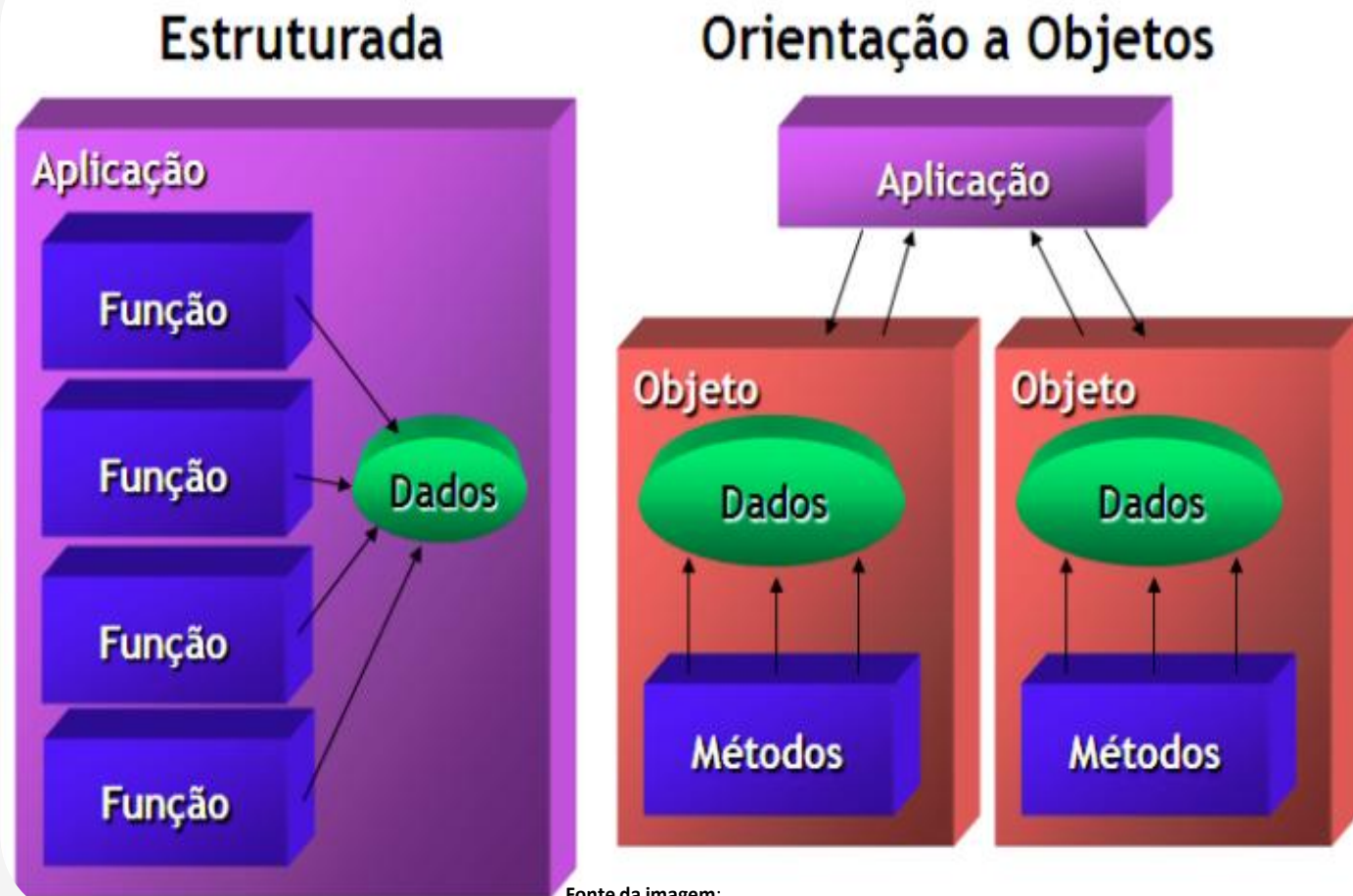
Programação Estruturada/Procedural/Imperativa

- ☐ Funcional;
- ☐ Lógico;
- ☐ Declarativa;
- ☐ Orientado a eventos;

Programação Orientada a Objetos (POO)

- ☐ Abstração;
- ☐ Herança/Polimorfismo;
- ☐ Encapsulamento;
- ☐ Concorrente;
- ☐ Reativa;

Estruturada vs. Orientação a Objetos



Fonte da imagem:

<https://docente.ifrn.edu.br/pedrobaesse/disciplinas/programacao-orientada-a-objetos/material-de-aula/aula-02-introducao-a-programacao-orientada-a-objeto>

Programação

Estruturada/Procedural

Chamada de procedimentos (ou funções) para manipulação de dados e variáveis;

Programação

Orientada a Objetos - POO

Estruturas de CLASSES com comportamentos;

Estruturada vs. Orientação a Objetos

Programação Estruturada	Programação Orientada a Objetos – POO
Procedimentos e funções	Métodos
Variáveis	Instâncias de atributos
Chamadas a procedimentos e funções	Método construtor e métodos de acesso;
Tipos de dados definidos de forma global	Atributos das Classes
-----	Herança
-----	Polimorfismo

Linguagens multiparadigma

```
print('Calculadora...\n')

while True:
    op = int(input('1 para inserir valores\n2 para efetuar operações\n3 para sair\nOpção: '))
    if op == 1:
        var1 = float(input('Digite um valor: '))
        var2 = float(input('Digite outro valor: '))
    elif op == 2:
        while True:
            op1 = int(input('\n1 para soma\n2 para subtrair\n3 para multiplicar\n4 para dividir\n5 para sair\nOpção: '))
            if op1 == 1:
                print('Soma: ', var1 + var2)
            elif op1 == 2:
                print('Subtração: ', var1 - var2)
            elif op1 == 3:
                print('Multiplicação: ', var1 * var2)
            elif op1 == 4:
                print('Divisão: ', var1 / var2)
            else:
                break
    else:
        break
```

Programação Estruturada Utilizando a linguagem **Python**

Fonte:

https://pt.wikipedia.org/wiki/Linguagem_de_programa%C3%A7%C3%A3o_multiparadigma

Linguagens multiparadigma

```
class Calculadora:
    def __init__(self):
        self.__var1 = None
        self.__var2 = None

    def lerValores(self):
        self.__var1 = float(input('Digite um valor: '))
        self.__var2 = float(input('Digite outro valor: '))

    def adicao(self):
        soma = self.__var1 + self.__var2
        return soma

    def subtracao(self):
        sub = self.__var1 - self.__var2
        return sub

    def multiplicacao(self):
        mult = self.__var1 * self.__var2
        return mult

    def divisao(self):
        div = self.__var1 / self.__var2
        return div
```

Classe - POO

```
from Calculadora import Calculadora

c = Calculadora()
while True:
    op = int(input('1 para setar valores\n2 para operações\n3 para sair\nOpção: '))
    if op == 1:
        c.lerValores()
    elif op == 2:
        while True:
            op1 = int(input('\n1 para soma\n2 para subtração\n3 para multiplicação\n4 para divisão\n5 para sair\nOpção: '))
            if op1 == 1:
                print(c.adicao())
            elif op1 == 2:
                print(c.subtracao())
            elif op1 == 3:
                print(c.multiplicacao())
            elif op1 == 4:
                print(c.divisao())
            else:
                break
    else:
        break
```

Orientação a Objetos (POO) Utilizando a linguagem Python

Fonte: https://pt.wikipedia.org/wiki/Linguagem_de_programa%C3%A7%C3%A3o_multiparadigma

Orientação a Objetos

Provê uma melhor
organização do código.
Contribui para o
reaproveitamento do código

POO - Vantagens

Pode não possuir o mesmo
desempenho de códigos
estruturados. Seus conceitos são
mais difíceis de compreender que
os conceitos da programação
estruturada.

POO - Desvantagens

Estruturada && Orientação a Objetos

- ✓ Na **Programação estruturada** observamos algumas vantagens como um controle mais eficaz quanto ao fluxo de execução do programa e a facilidade de compreender o código quando o mesmo é analisado.
- ✓ Na **POO** temos como vantagens a reutilização de código e a melhor organização do código do programa.

Em ambas os paradigmas existem características bem peculiares nas suas definições, onde dependendo da complexidade da solução a ser implementada uma pode ser mais viável que a outra.

De forma que podemos entender que ao invés de concorrer, **se usadas de maneira inteligente pelos programadores elas se complementam!**



Pilares - POO

Princípio da Abstração:

O conceito de abstração consiste em esconder os detalhes de algo, no caso, os detalhes desnecessários. Na vida, utilizamos abstrações o tempo todo. Em tudo que não sabemos como funciona nos bastidores, pode ser considerado uma abstração.

- ☐ Encapsulamento;
- ☐ Herança;
- ☐ Composição;
- ☐ Polimorfismo;



<https://materialpublic.imd.ufrn.br/curso/disciplina/2/8/1/4>

Instanciando uma classe com Python

```
class Produto:
    def __init__(self, descricao, preco):
        self.descricao=descricao
        self.preco=preco
    def verProduto(self):
        print(f'{self.descricao} por apenas R${self.preco:.2f}')
```

```
from classe import Produto
notebook = Produto('Notebook Dell i7', 5800.90)
input('Pressione qualquer tecla para ver a oferta: ')
notebook.verProduto()
print()
```

```
Pressione qualquer tecla para ver a oferta:
Notebook Dell i7 por apenas R$5800.90
```

Desafio Calculadora:

Fazer uma calculadora que receba **dois valores** e retorne o resultado de **adição**, **subtração**, **multiplicação** ou **divisão** destes dois valores;

Resposta do desafio Python

```
class Calculadora:
    def __init__(self, valor1, valor2):
        self.valor1=valor1
        self.valor2 =valor2
    def soma(self):
        return self.valor1 + self.valor2
    def sub(self):
        return self.valor1 - self.valor2
    def mult(self):
        return self.valor1 * self.valor2
    def div(self):
        return self.valor1 / self.valor2
```

```
from classe import Calculadora
valor1 = int(input('Informe o 1º valor: '))
valor2 = int(input('Informe o 2º valor: '))
calc = Calculadora(valor1, valor2)
print('Soma: ')
print(calc.soma())
print('---')
print('Subtração: ')
print(calc.sub())
print('---')
print('Multiplicação: ')
print(calc.mult())
print('---')
print('Divisão: ')
print(calc.div())
print('---')
```

```
Informe o 1º valor: 100
Informe o 2º valor: 50
Soma:
150
---
Subtração:
50
---
Multiplicação:
5000
---
Divisão:
2.0
```

Mão na massa... Encapsulamento com o Python

```
class AssinaturaTv:
    def __init__(self):
        self.canaisDisponiveis=(2, 5, 7, 15, 30, 50)
        self.__canalAtivo = 7
        self.volume=10

    @property
    def canalAtivo(self):
        return(self.__canalAtivo)

    @canalAtivo.setter
    def canalAtivo(self, canal):
        try:
            index=self.canaisDisponiveis.index(canal)
            self.__canalAtivo=canal
            print(f'O canal ativo agora é: {canal}')
        except:
            index=-1
            print(f'O canal {canal} não está disponível')
```

```
from classe import AssinaturaTv
tv = AssinaturaTv()
print(tv)
print('Canais disponíveis')
print(tv.canaisDisponiveis)
print('---')
print('Canal Ativo')
print(tv.canalAtivo)
print('---')
tv.canalAtivo = 15
print(tv.canalAtivo)
print('---')
tv.canalAtivo = 25
print('Canal Ativo')
print(tv.canalAtivo)
print('---')
```

```
Canais disponíveis
(2, 5, 7, 15, 30, 50)
---
Canal Ativo
7
---
O canal ativo agora é: 15
15
---
O canal 25 não está disponível
Canal Ativo
15
```

Desafio Produto:

Construir uma classe Produto que com atributos **descrição**, **precoCusto** e **precoVenda**. O atributo **precoVenda** deve ser protegido e ter um **método GET e SET** alterando o valor apenas se o novo valor for maior que o **precoCusto**;

Resposta do desafio

```
class Produto:
    def __init__(self, descricao, precoCusto, precoVenda):
        self.descricao=descricao
        self.precoCusto = precoCusto
        self.__precoVenda=precoVenda
    @property
    def precoVenda(self):
        return(self.__precoVenda)
    @precoVenda.setter
    def precoVenda(self, valor):
        if(valor>=self.precoCusto):
            self.__precoVenda=valor
            print(f'Preço de venda alterado para R${valor}')
        else:
            print('O preço de venda não pode ser alterado.')
```

```
from classe import Produto
notebook = Produto('Notebook Dell i7', 3800, 5800)
print("Mudar preço para 1000")
notebook.precoVenda=100
print("Mudar preço para 5100")
notebook.precoVenda=5100
print('----')
```

```
Mudar preço para 1000
O preço de venda não pode ser alterado.
Mudar preço para 5100
Preço de venda alterado para R$5100
----
```


Exemplo de Composição Agregação

```
class CicloMensal:
    def __init__(self, mes, ano):
        self.mes=mes
        self.ano=ano
        self.lancamentos=[]
    def addLancamento(self, descricao, valor):
        lancamento=Lancamento(descricao, valor)
        self.lancamentos.append(lancamento)

    def calcularTotal(self):
        total=0
        for l in self.lancamentos:
            print(f'{l.descricao} | R${l.valor:.2f}')
            total+=l.valor
        return total

class Lancamento:
    def __init__(self, descricao, valor):
        self.descricao=descricao
        self.valor=valor
```

```
from classe import *
abril=CicloMensal( 'Abril', 22)
abril.addLancamento('Salario', 4000)
abril.addLancamento('Cartão Alimentação', 600)
abril.addLancamento('Trabalho Extra', 2000)
abril.addLancamento('Supermercado', -2000)
abril.addLancamento('Cartão de credits', -1800)
print('-----')
print(f'Lançamentos: {abril.mes}/{abril.ano}')
print('---')
total=abril.calcularTotal()
print('---')
print(f'Total: R${total}')
```

```
Lançamentos: Abril/22
---
Salario | R$4000.00
Cartão Alimentação | R$600.00
Trabalho Extra | R$2000.00
Supermercado | R$-2000.00
Cartão de credits | R$-1800.00
---
Total: R$2800
```

Desafio CarrinhoCompras:

Construir uma classe **CarrinhoCompras** que tenha como **atributos**: **data da compra**, um **objeto Cliente** composto por **nome e endereço** e o atributo **produtos** onde serão agregado **N objetos da classe produto**. A classe produto deve ter os atributos **descrição e preço**. A classe Carrinho de compras deve ter um método para somar os preços de todos os produtos agregados e exibir o total.

Resposta do desafio

```
class CarrinhoCompas:
    def __init__(self, data, nome, endereco):
        self.data=data
        self.cliente=Cliente(nome, endereco)
        self.produtos=[]
    def addProduto(self,descricao, preco):
        produto=Produto(descricao, preco)
        self.produtos.append(produto)
    def calcularTotal(self):
        total=0
        for p in self.produtos:
            print(p.descricao, p.preco)
            total+=p.preco
        return total
```

```
class Cliente:
    def __init__(self, nome, endereco):
        self.nome=nome
        self.endereco=endereco
class Produto:
    def __init__(self, descricao, preco):
        self.descricao=descricao
        self.preco = preco
```

```
from classe import *
data="30/04/2022"
nome="Joaquim da Silva"
endereco="Rua dos Pombos, 000"
carrinho=CarrinhoCompas( data, nome, endereco)
carrinho.addProduto('Notebook Dell i7', 5800)
carrinho.addProduto('SmartPhone', 2800)
carrinho.addProduto('XBox', 3400)
print(f'Carrinho: {carrinho.data}')
print('Cliente',carrinho.cliente.nome)
print('Endereco',carrinho.cliente.endereco)
print('-----')
total=carrinho.calcularTotal()
print(f'Total R${total}')
print('---')
```

```
Carrinho: 30/04/2022
Cliente Joaquim da Silva
Endereco Rua dos Pombos, 000
-----
Notebook Dell i7 5800
SmartPhone 2800
XBox 3400
Total R$12000
```